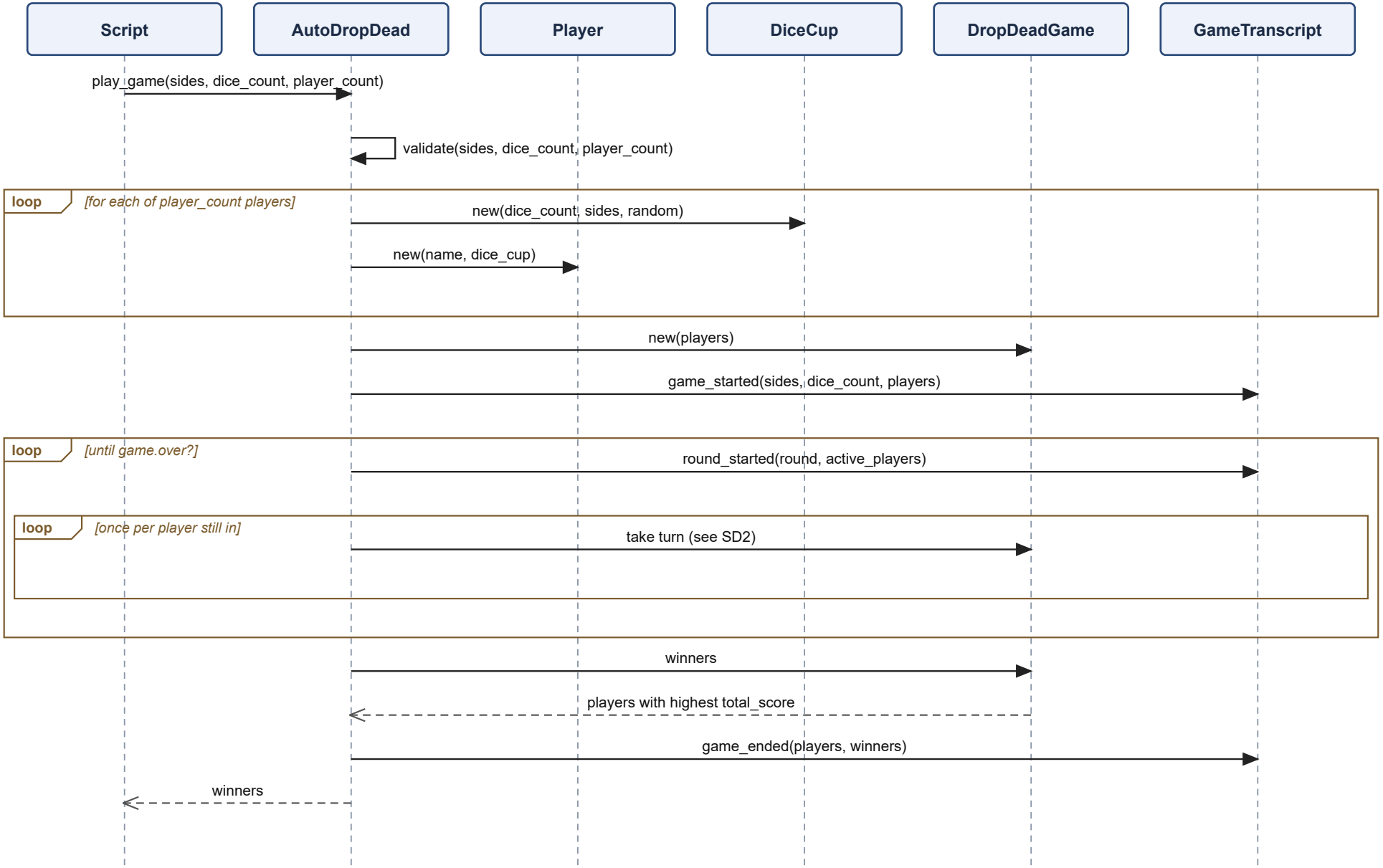
SD1 - Use Case: Play Game

Purpose: run a complete automatic game from setup to announcing the winner.

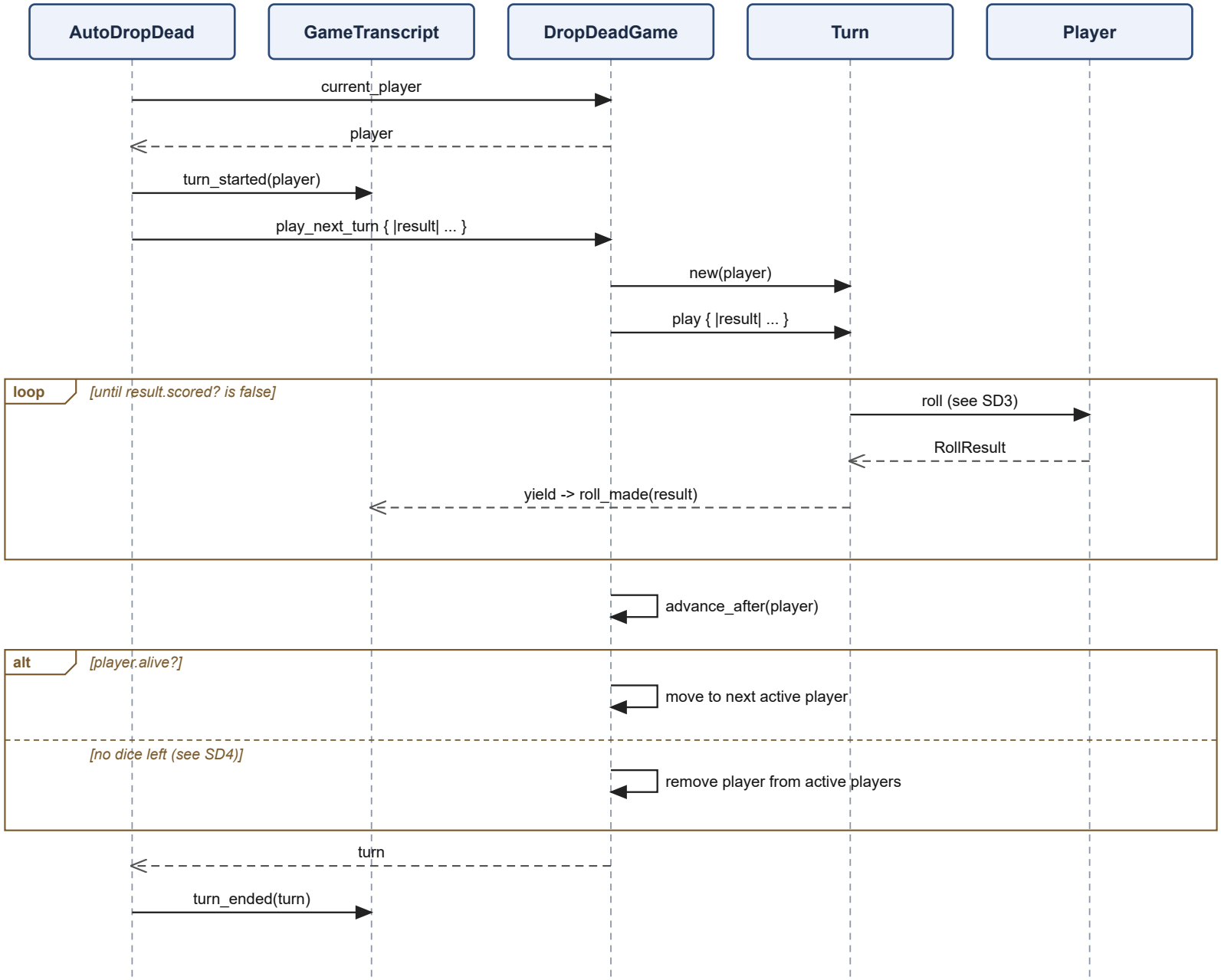
Source: generated by Claude (Anthropic AI assistant) for this assignment.



SD2 - Use Case: Take Turn

Purpose: let the current player roll until a 2 or 5 appears, then pass play on.

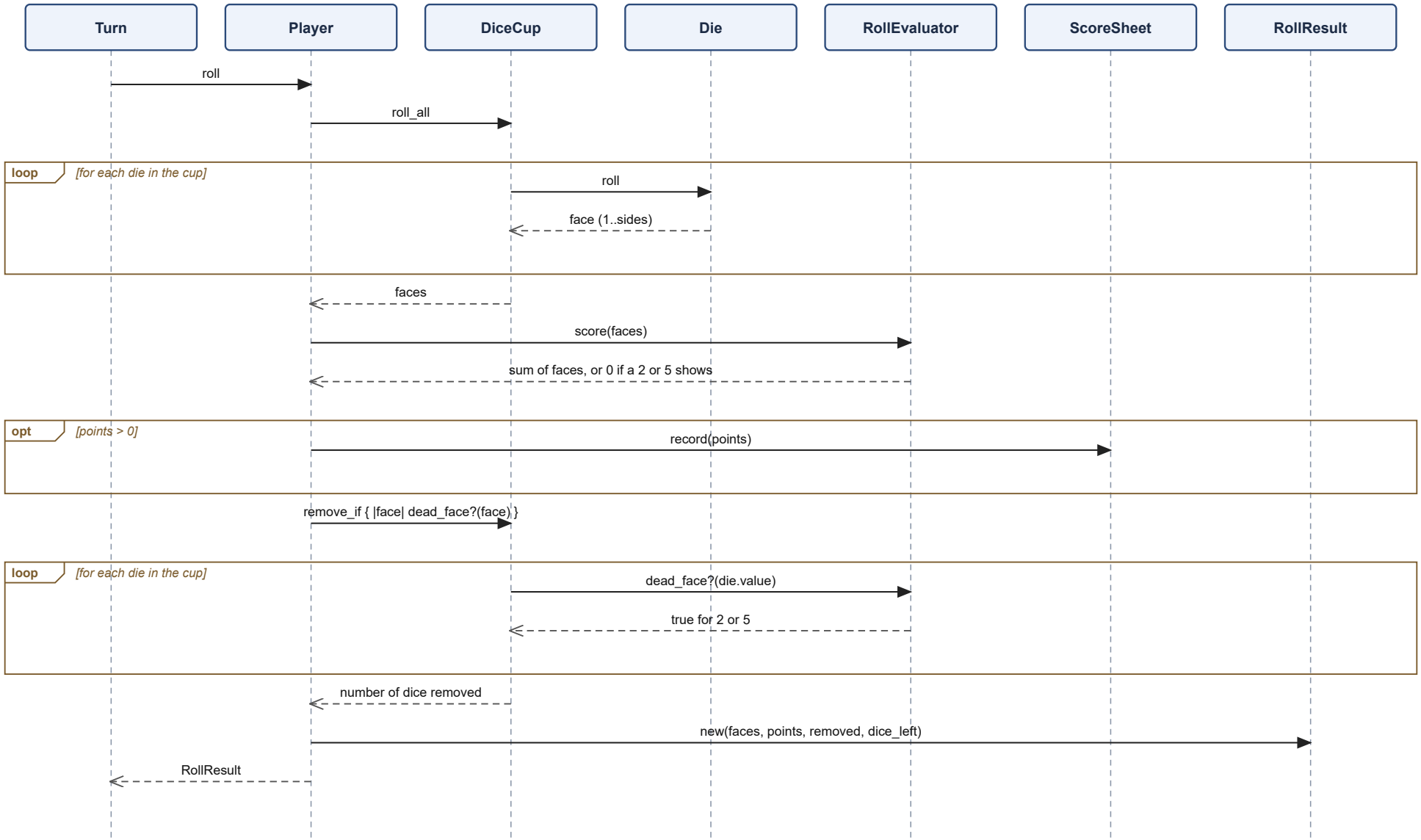
Source: generated by Claude (Anthropic AI assistant) for this assignment.



SD3 - Use Case: Roll Dice

Purpose: roll a player's remaining dice once, score the roll and remove any 2s and 5s.

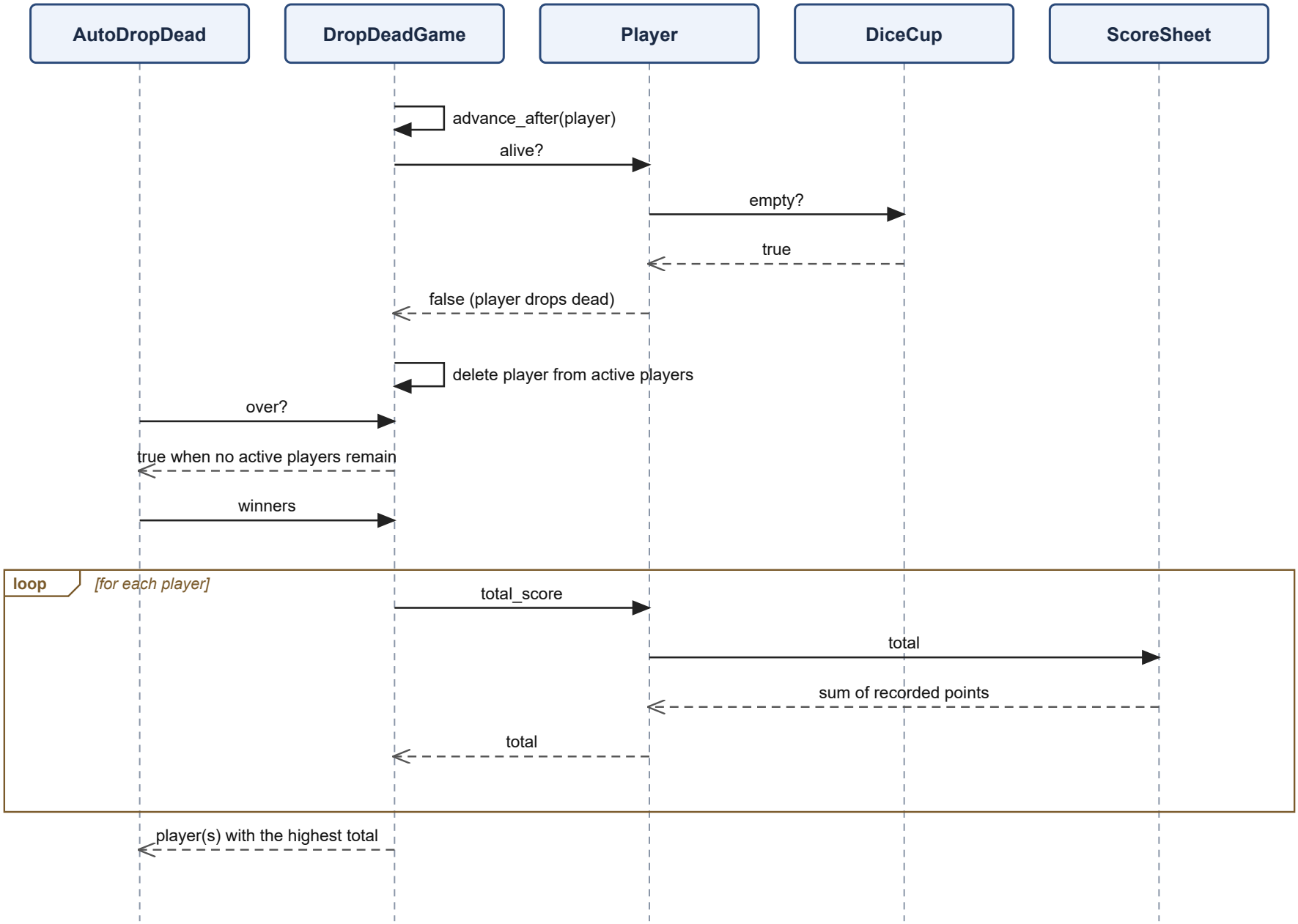
Source: generated by Claude (Anthropic AI assistant) for this assignment.



SD4 - Use Case: Player Drops Dead / Determine Winner

Purpose: remove a player with no dice left, end the game when nobody is left, and pick the winner.

Source: generated by Claude (Anthropic AI assistant) for this assignment.



CIS*3260 Assignment 1 - Drop Dead Dice Game

Sequence Diagrams

- The diagrams are vector graphics on pages 1-4 of sequence_class_descriptions.pdf.
- Attribution: all four diagrams were generated by Claude (Anthropic's AI assistant).
- SD1 (page 1)
  - Use case: Play Game
  - Purpose: run a complete automatic game, from validating the settings and creating the players to announcing the winner.
- SD2 (page 2)
  - Use case: Take Turn
  - Purpose: let the current player keep rolling until a 2 or 5 appears, then pass play to the next active player.
- SD3 (page 3)
  - Use case: Roll Dice
  - Purpose: roll a player's remaining dice once, score the roll, record the points and remove every die showing a 2 or 5.
- SD4 (page 4)
  - Use case: Player Drops Dead / Determine Winner
  - Purpose: remove a player with no dice left, end the game when no players remain, and choose the player(s) with the most points.

Die

- Responsibility: produce a random face value for one die.
- Instance variables
  - @sides: number of faces on the die (at least 6)
  - @random: random number generator used to roll
  - @value: face showing after the last roll (nil before rolling)
- Public methods
  - initialize(sides, random = Random.new): creates a die; raises ArgumentError if sides < 6. Responsibility: build a valid die.
  - roll: rolls the die and returns the face (1..sides). Responsibility: generate a new random face.
  - sides: Responsibility: report how many sides the die has.
  - value: Responsibility: report the face currently showing.
  - to_s: Responsibility: give a printable form of the face showing.

DiceCup

- Responsibility: manage the dice a player still owns.
- Instance variables
  - @dice: the dice the player has not yet lost
- Public methods
  - initialize(dice_count, sides, random = Random.new): fills the cup with dice_count dice. Responsibility: build the player's starting dice.
  - roll_all: rolls every die and returns the faces. Responsibility: roll all remaining dice together.
  - remove_if { |face| ... }: removes dice whose face makes the block true and returns how

many were removed. Responsibility: take dice out of the cup.
  - count: Responsibility: report how many dice are left.
  - empty?: Responsibility: report whether the cup has no dice left.

RollEvaluator

- Responsibility: apply the Drop Dead scoring rules to a roll.
- Instance variables
  - none (the losing faces are the constant DEAD_FACES = [2, 5])
- Public methods
  - dead_face?(face): true when the face is a 2 or 5. Responsibility: decide whether a single die is lost.
  - contains_dead_face?(faces): true when any face is a 2 or 5. Responsibility: decide whether a roll ends the turn.
  - score(faces): sum of the faces, or 0 if the roll contains a 2 or 5. Responsibility: compute the points a roll is worth.

ScoreSheet

- Responsibility: record and total a player's points (the player's paper).
- Instance variables
  - @entries: points written down, one entry per scoring roll
- Public methods
  - initialize: starts an empty sheet. Responsibility: create a blank score sheet.
  - record(points): writes the points from one roll. Responsibility: store a score.
  - total: adds up every entry. Responsibility: tally the points.
  - entries: returns a copy of the entries. Responsibility: expose the score history without allowing changes.

RollResult

- Responsibility: describe the outcome of a single roll.
- Instance variables
  - @faces: faces that came up
  - @points: points earned by the roll
  - @dice_removed: number of dice lost (2s and 5s)
  - @dice_left: dice the player still has after the roll
- Public methods
  - initialize(faces, points, dice_removed, dice_left): builds a frozen record of the roll. Responsibility: capture a roll's outcome.
  - faces, points, dice_removed, dice_left: Responsibility: report one fact about the roll.
  - scored?: true when no dice were removed, so the player rolls again. Responsibility: say whether the turn continues.
  - dropped_dead?: true when the roll took the player's last die. Responsibility: say whether the player is out.

Player

- Responsibility: represent one participant and carry out their rolls.
- Instance variables

  - @name: display name, e.g. "Player 1"
  - @dice_cup: the dice the player still owns
  - @score_sheet: the points the player has written down
  - @evaluator: the rules used to judge each roll
- Public methods
  - initialize(name, dice_cup, evaluator = RollEvaluator.new): creates a player with dice and a blank score sheet. Responsibility: set up a participant.
  - roll: rolls the remaining dice once, records any points and removes 2s and 5s; returns a RollResult. Responsibility: perform one roll for this player.
  - dice_left: Responsibility: report how many dice the player has.
  - alive?: true while the player has at least one die. Responsibility: report whether the player is still in the game.
  - total_score: Responsibility: report the player's points.
  - name, score_sheet, to_s: Responsibility: identify the player or expose their sheet.

Turn

- Responsibility: play out one player's turn.
- Instance variables
  - @player: whose turn it is
  - @rolls: every roll made during the turn
- Public methods
  - initialize(player): prepares a turn. Responsibility: start a turn.
  - play { |result| ... }: rolls until a roll contains a 2 or 5, yielding each result. Responsibility: apply the "keep rolling until a 2 or 5" rule.
  - rolls: Responsibility: report the rolls made this turn.
  - points: Responsibility: report the points earned this turn.
  - player: Responsibility: identify whose turn it is.

DropDeadGame

- Responsibility: track the players still in the game and whose turn it is.
- Instance variables
  - @players: everyone who started the game
  - @active_players: players who have not dropped dead
  - @current_index: position in @active_players of the next player to take a turn
- Public methods
  - initialize(players): starts a game; raises ArgumentError if there are no players. Responsibility: set up the table.
  - players: Responsibility: report everyone in the game.
  - active_players: Responsibility: report who is still playing.
  - current_player: Responsibility: report whose turn is next.
  - over?: true when no players are left. Responsibility: decide whether the game has ended.
  - play_next_turn { |result| ... }: plays the current player's turn, removes them if they drop dead and moves to the next player; returns the Turn. Responsibility: advance the game by one turn.
  - winners: the player(s) with the highest total. Responsibility: decide who won.

GameTranscript

- Responsibility: write a readable account of the game as it happens.
- Instance variables
  - @output: where the transcript is written (default $stdout)
- Public methods
  - initialize(output = $stdout): Responsibility: choose where the transcript goes.
  - game_started(sides, dice_count, players): Responsibility: print the game settings.
  - round_started(round, players): Responsibility: print a round header.
  - turn_started(player): Responsibility: print the player's dice and points before their turn.
  - roll_made(result): Responsibility: print one roll and what it did.
  - turn_ended(turn): Responsibility: print the turn summary, or that the player dropped dead.
  - game_ended(players, winners): Responsibility: print the final scores and the winner(s).

AutoDropDead

- Responsibility: play a complete game automatically, with the computer controlling every player.
- Instance variables
  - @transcript: reports the game as it is played
  - @random: shared random number generator for every die
- Public methods
  - initialize(output = $stdout, random = Random.new): Responsibility: configure the output and source of randomness.
  - play_game(sides, dice_count, player_count): validates the settings (sides >= 6, dice_count >= 2, player_count >= 1, otherwise ArgumentError), creates the players, plays rounds until everyone has dropped dead, prints the transcript and returns the winner(s). Responsibility: run one full automatic game.